

Refinamento de prompts

Pedi à IA o mesmo programa três vezes. Só mudei o jeito de pedir.

ACERTOS EM 10 PERGUNTAS

P1 0

P2 6

P3 10

Levar as vendas do dia para o painel

Todo dia, o sistema da loja gera um arquivo com as vendas. Um programa precisa levar essas vendas para o painel de faturamento. Esse tipo de programa se chama ETL: extrai, transforma e carrega.

A ARMADILHA

O programa pode dizer “deu certo” e mesmo assim deixar os números errados. “Rodou” não prova nada.

COMO CONFERI

10 perguntas de sim ou não, escritas antes de começar.

Cada programa rodou com os mesmos dois arquivos, e com um deles duas vezes.

O ARQUIVO DO DIA 02/09, COMO CHEGA

```
id_venda;data;...;qtd;valor;status;atualizado_em
1001;02/09/2026;...;2;18,90;FATURADA;02/09/2026 09:12
1002;02/09/2026;...;1;3.299,00;FATURADA;02/09/2026 15:47
1003;02/09/2026;...;4;1.234,56;FATURADA;02/09/2026 10:40
1002;02/09/2026;...;1;3.499,00;FATURADA;02/09/2026 10:05
1004;02/09/2026;...;3;79,90; cancelada ;02/09/2026 11:20
1005;02/09/2026;...;;149,90;FATURADA;02/09/2026 13:02
1006;02/09/2026;...;2;899,99;FATURADA;02/09/2026 16:30
Total de registros: 7
```

venda repetida · vírgula nos centavos · status com espaço ·
quantidade vazia · linha de total

Pedir sem explicar nada

0 PEDIDO, COMPLETO

Crie um script ETL em Python que carregue um CSV de vendas em um banco de dados.

A RESPOSTA DA IA: SCRIPT COMPLETO (etl_r1.py)

```

1 import logging
2 import sqlite3
3
4 import pandas as pd
5
6 logging.basicConfig(
7     level=logging.INFO, format="%(asctime)s - %(levelname)s - %(message)s"
8 )
9
10
11 def extrair(caminho_csv: str) -> pd.DataFrame:
12     """Lê o arquivo CSV de vendas."""
13     logging.info("Extraindo dados de %s", caminho_csv)
14     return pd.read_csv(caminho_csv)
15
16
17 def transformar(df: pd.DataFrame) -> pd.DataFrame:
18     """Limpa e padroniza os dados."""
19     logging.info("Transformando %d linhas", len(df))
20     df.columns = df.columns.str.strip().str.lower().str.replace(" ", "_")
21     df = df.drop_duplicates()
22     df = df.dropna()
23
24     if "data" in df.columns:
25         df["data"] = pd.to_datetime(df["data"], errors="coerce")

```

```

26
27     if {"quantidade", "preco"}.issubset(df.columns):
28         df["total"] = df["quantidade"] * df["preco"]
29
30     return df
31
32
33 def carregar(
34     df: pd.DataFrame, caminho_banco: str, tabela: str = "vendas"
35 ) -> None:
36     """Grava os dados no banco SQLite."""
37     logging.info("Carregando %d linhas na tabela %s", len(df), tabela)
38     with sqlite3.connect(caminho_banco) as conn:
39         df.to_sql(tabela, conn, if_exists="replace", index=False)
40
41
42 def executar_etl(caminho_csv: str, caminho_banco: str) -> None:
43     df = extrair(caminho_csv)
44     df = transformar(df)
45     carregar(df, caminho_banco)
46     logging.info("ETL concluído com sucesso")
47
48
49 if __name__ == "__main__":
50     executar_etl("vendas.csv", "vendas.db")

```

Travou. E, corrigido, mentiu.

RODANDO COM O ARQUIVO REAL

```
INFO - Extraindo dados de vendas_2026-09-02.csv
UnicodeDecodeError: 'utf-8' codec can't decode
byte 0xe3 in position 96
```

COM OS ACENTOS DO ARQUIVO CORRIGIDOS

```
INFO - Transformando 8 linhas
INFO - Carregando 7 linhas na tabela vendas
INFO - ETL concluído com sucesso
```

```
tabela vendas: 7 linhas, 1 coluna
90;FATURADA;02/09/2026 09:12
00;FATURADA;02/09/2026 15:47
```

O QUE ACONTECEU • 0 DE 10

- linha 14 Lê o arquivo sem saber dos acentos nem do ponto e vírgula. Travou; com os acentos corrigidos, cortou cada linha na vírgula dos centavos.
- linha 22 Linha com campo vazio some sem aviso.
- linha 25 Lê 02/09 como 9 de fevereiro.
- linha 27 Procura a coluna “preco”, que não existe: o total nunca é calculado.
- linha 39 “replace”: cada carga apaga os dias anteriores.

Faltou no pedido: como o arquivo é, as regras da loja e como o programa é usado.

Mostrar como o arquivo é

0 PEDIDO, COMPLETO

Atue como engenheiro de dados com experiência em Python e pandas.

Contexto: uma loja exporta do ERP, todo dia, um arquivo vendas_AAAA-MM-DD.csv. Preciso carregar esses arquivos num banco SQLite (vendas.db) que alimenta um dashboard de faturamento.

O arquivo vem assim (primeiras linhas):

```
id_venda;data;cliente;produto;quantidade;valor_unitario;status;atualizado_em
1001;02/09/2026;João Araújo;Caderno universitário;2;18,90;FATURADA;02/09/2026 09:12
1003;02/09/2026;Pedro Sá;Cadeira de escritório;4;1.234,56;FATURADA;02/09/2026 10:40
```

- codificação latin-1, separador ";"
- valores com vírgula decimal e ponto de milhar;
- datas no formato dia/mês/ano.

Regras de negócio:

- vendas com status CANCELADA não entram na tabela;
- se o mesmo id_venda aparecer mais de uma vez, vale a linha com atualizado_em mais recente;
- valor_total = quantidade × valor_unitario.

Tabela de destino "vendas": id_venda, data_venda, cliente, produto, quantidade, valor_unitario, valor_total.

Escreva o script ETL. Sou estudante de Ciência da Computação e já uso pandas no básico.

Formato: funções separadas para extrair, transformar e carregar, e uma função executar(caminho_csv, caminho_banco).

Restrições: apenas pandas e sqlite3.

Critérios: código legível, com comentários curtos onde a regra de negócio aparece.

NOVO EM RELAÇÃO AO PROMPT 1

papel

contexto

formato do arquivo

regras da loja

público

formato da resposta

restrições

critérios

O script que veio

A RESPOSTA DA IA: SCRIPT COMPLETO (etl_r2.py)

```

1 import sqlite3
2
3 import pandas as pd
4
5 COLUNAS_DESTINO = ["id_venda", "data_venda", "cliente", "produto",
6                     "quantidade", "valor_unitario", "valor_total"]
7
8
9 def extrair(caminho_csv):
10     # latin-1 e ";" vêm do ERP; decimal e thousands convertem "1.234,56"
11     return pd.read_csv(
12         caminho_csv,
13         sep=";",
14         encoding="latin-1",
15         decimal=",",
16         thousands=".",
17         dtype={"id_venda": str},
18     )
19
20
21 def transformar(df):
22     df = df.copy()
23
24     # Regra: canceladas ficam de fora (normaliza caixa e espaços)
25     df["status"] = df["status"].str.strip().str.upper()
26     df = df[df["status"] != "CANCELADA"]
27
28
29 # Datas em dia/mês/ano
30 df["data_venda"] = pd.to_datetime(df["data"], format="%d/%m/%Y").dt.date
31 df["atualizado_em"] = pd.to_datetime(
32     df["atualizado_em"], format="%d/%m/%Y %H:%M"
33 )
34
35 # Regra: se o id se repete, vale a versão mais recente
36 df = df.sort_values("atualizado_em").drop_duplicates(
37     "id_venda", keep="last"
38 )
39
40 # Regra: valor_total = quantidade x valor_unitario
41 df["valor_total"] = (df["quantidade"] * df["valor_unitario"]).round(2)
42
43 return df[COLUNAS_DESTINO]
44
45 def carregar(df, caminho_banco):
46     with sqlite3.connect(caminho_banco) as conn:
47         df.to_sql("vendas", conn, if_exists="append", index=False)
48
49
50 def executar(caminho_csv, caminho_banco):
51     df = extrair(caminho_csv)
52     df = transformar(df)
53     carregar(df, caminho_banco)
54     return len(df)

```

Depois do código, a IA avisou: “rodar o mesmo arquivo duas vezes repete as vendas; se isso for um problema, dá para usar uma chave primária.”

Rodou sem erro. E errou quatro vezes.

```
NO BANCO, DEPOIS DA CARGA DE 02/09
```

```
id_venda      quantidade  valor_total
1005          None      None
Total de registros: 7  None      None
```

```
programa informou: 6 vendas carregadas
```

```
DEPOIS DE RODAR 02/09 DE NOVO E CARREGAR 03/09
```

```
15 linhas para 9 vendas diferentes
venda 1003 (cancelada em 03/09): 2 linhas
```

O QUE ACONTECEU • 6 DE 10

linhas 11-18 Agora lê acentos, separador e centavos.

A linha de total virou uma venda: o exemplo do pedido não mostrava essa linha.

linha 40 Quantidade vazia vira total vazio, sem aviso. O painel somaria menos.

linha 26 Só tira a cancelada do arquivo do dia. A 1003, cancelada depois, ficou.

linha 47 “append” acrescenta: rodar de novo duplica.

Nota da aula: P1 16 → P2 27 (de 35). Nenhuma das quatro falhas apareceu como erro na tela.

Contar como o programa é usado

O QUE AINDA PRECISAVA MELHORAR

- 01 O exemplo só mostrava vendas “bonitas”. Linha de total e campo vazio não apareciam.
- 02 Eu não disse como o programa é usado: todo dia, roda de novo se falhar, pode trazer mudanças de ontem.
- 03 A IA viu o problema de rodar de novo, mas só avisou.
- 04 “Código legível” não dá para conferir e não evitou nenhuma falha.

HIPÓTESE

Se a IA souber como o arquivo chega de verdade e como o programa é usado no dia a dia, ela vai tratar os casos que o programa 2 deixou passar sem avisar.

ENTROU

- como o programa é usado
- linha de total e campo vazio
- lista de linhas com defeito
- dúvidas antes do código

SAIU

- as linhas de exemplo
- “código legível”

O pedido, completo

PAPEL

Atue como engenheiro de dados com experiência em pipelines de carga incremental em Python.

CONTEXTO

Uma loja exporta do ERP, todo dia, um arquivo vendas_AAAA-MM-DD.csv com as vendas criadas ou alteradas naquele dia. O script roda agendado e, se falhar, é executado de novo com o mesmo arquivo. O banco SQLite (vendas.db) alimenta um dashboard de faturamento.

ARQUIVO DE ORIGEM

- codificação latin-1, separador ";"
- valores com vírgula decimal e ponto de milhar ("1.234,56");
- datas dd/mm/aaaa e atualizado_em dd/mm/aaaa hh:mm;
- a última linha é um rodapé do ERP: "Total de registros: N";
- às vezes vem linha com campo obrigatório vazio.

Colunas: id_venda;data;cliente;produto;quantidade;valor_unitario;status;atualizado_em

REGRAS

- R1: status CANCELADA (qualquer caixa, com ou sem espaços) não fica na tabela vendas, inclusive se a venda já tiver sido carregada antes;
- R2: id_venda repetido: vale o atualizado_em mais recente;
- R3: valor_total = quantidade × valor_unitario, com 2 casas;
- R4: linha inválida não entra em vendas e vai para a tabela rejeitados, com o motivo;
- R5: rodar o mesmo arquivo de novo não pode duplicar nem perder dados.

DESTINO

vendas(id_venda PK, data_venda, cliente, produto, quantidade, valor_unitario, valor_total, atualizado_em)
rejeitados(arquivo, linha, motivo)

PÚBLICO

Estudante de Ciência da Computação que usa pandas no básico.

TAREFA

1. Antes do código, liste as ambiguidades e a decisão em cada uma.
2. Escreva o script, com a função executar(caminho_csv, caminho_banco).
3. Explique em até cinco linhas como o script garante a R5.

RESTRIÇÕES

- apenas pandas e sqlite3;
- não usar if_exists="replace";
- a carga de um arquivo é tudo ou nada (uma transação).

CRITÉRIOS DE QUALIDADE

- toda linha do arquivo termina em vendas, em rejeitados ou numa contagem de descartes (canceladas, duplicatas, rodapé);
- executar duas vezes o mesmo arquivo deixa o banco igual.

Antes do código, as dúvidas

AMBIGUIDADES QUE A IA APONTOU

#	PONTO NÃO DEFINIDO	DECISÃO ADOTADA
A1	O arquivo do dia pode trazer venda de dia anterior (alteração)	Tratar como atualização pelo id_venda (upsert)
A2	Quantidade zero ou negativa	Rejeitar, mesmo não sendo campo vazio
A3	Reprocessar o arquivo gera os mesmos rejeitados de novo	Apagar os rejeitados daquele arquivo antes de gravar
A4	Cancelada que nunca foi carregada	Só entra na contagem de canceladas
A5	Arquivo antigo reprocessado depois de um mais novo	Só sobrescrever se atualizado_em for igual ou mais recente

COMO A IA DISSE QUE EVITA DUPLICAR (R5)

id_venda é chave primária e a gravação é um upsert, então a mesma venda nunca vira duas linhas. Os rejeitados do arquivo são apagados e regravados. Tudo roda numa transação: se algo falhar no meio, nada fica pela metade.

O QUE A IA DISSE SOBRE O RESUMO

A função executar devolve um resumo com as contagens (lidas, rodapé, rejeitadas, canceladas, duplicatas, carregadas), para conferir que toda linha do arquivo teve um destino.

Script completo, linhas 1 a 74

A RESPOSTA DA IA: etl_r3.py

```

1  """
2  Carga incremental das vendas diárias do ERP para o SQLite.
3
4  Uso: executar("vendas_2026-09-02.csv", "vendas.db")
5  """
6  import sqlite3
7  from pathlib import Path
8
9  import pandas as pd
10
11  COLUNAS = ["id_venda", "data", "cliente", "produto", "quantidade",
12            "valor_unitario", "status", "atualizado_em"]
13
14  DDL = """
15  CREATE TABLE IF NOT EXISTS vendas (
16      id_venda      TEXT PRIMARY KEY,
17      data_venda    TEXT NOT NULL,
18      cliente       TEXT NOT NULL,
19      produto       TEXT NOT NULL,
20      quantidade    INTEGER NOT NULL,
21      valor_unitario REAL NOT NULL,
22      valor_total   REAL NOT NULL,
23      atualizado_em TEXT NOT NULL
24  );
25  CREATE TABLE IF NOT EXISTS rejeitados (
26      arquivo TEXT NOT NULL,
27      linha   INTEGER NOT NULL,
28      motivo  TEXT NOT NULL
29  );
30  """
31
32  # Só sobrescreve se a versão que chega for igual ou mais nova (A5)
33  UPSERT = """
34  INSERT INTO vendas VALUES (:id_venda, :data_venda, :cliente, :produto,
35      :quantidade, :valor_unitario, :valor_total, :atualizado_em)
36  ON CONFLICT(id_venda) DO UPDATE SET
37      data_venda      = excluded.data_venda,

```

```

38      cliente        = excluded.cliente,
39      produto        = excluded.produto,
40      quantidade      = excluded.quantidade,
41      valor_unitario  = excluded.valor_unitario,
42      valor_total     = excluded.valor_total,
43      atualizado_em   = excluded.atualizado_em
44  WHERE excluded.atualizado_em >= vendas.atualizado_em
45  """
46
47  APAGAR_CANCELADA = (
48      "DELETE FROM vendas WHERE id_venda = ? AND atualizado_em <= ?"
49  )
50
51
52  def numero_br(texto):
53      """'1.234,56' -> 1234.56"""
54      return float(texto.replace(".", "").replace(",", "."))
55
56
57  def extrair(caminho_csv):
58      # tudo como texto: a conversão acontece na transformação, linha a linha
59      df = pd.read_csv(caminho_csv, sep=";", encoding="latin-1",
60                      dtype=str, keep_default_na=False)
61      df["linha"] = df.index + 2 # linha no arquivo (cabeçalho é a 1)
62      return df
63
64
65  def transformar(df, arquivo):
66      resumo = {"lidas": len(df), "rodape": 0, "rejeitadas": 0,
67              "canceladas": 0, "duplicatas": 0, "carregadas": 0}
68      rejeitados, validas = [], []
69
70      for reg in df.to_dict("records"):
71          if reg["id_venda"].strip().lower().startswith("total de registros"):
72              resumo["rodape"] += 1
73          continue
74

```

Script completo, linhas 75 a 152

A RESPOSTA DA IA: etl_r3.py

```

75     vazios = [c for c in COLUNAS if not str(reg.get(c) or "").strip()]
76     if vazios:
77         rejeitados.append(
78             (arquivo, reg["linha"], "campo vazio: " + ", ".join(vazios))
79         )
80         continue
81
82     try:
83         quantidade = int(reg["quantidade"])
84         valor_unitario = numero_br(reg["valor_unitario"])
85         data_venda = pd.to_datetime(reg["data"], format="%d/%m/%Y")
86         atualizado_em = pd.to_datetime(
87             reg["atualizado_em"], format="%d/%m/%Y %H:%M"
88         )
89     except ValueError:
90         rejeitados.append((arquivo, reg["linha"], "formato inválido"))
91         continue
92
93     if not reg["id_venda"].strip().isdigit() or quantidade <= 0:
94         rejeitados.append(
95             (arquivo, reg["linha"], "id_venda ou quantidade inválidos")
96         )
97         continue
98
99     validas.append({
100         "id_venda": reg["id_venda"].strip(),
101         "data_venda": data_venda.strftime("%Y-%m-%d"),
102         "cliente": reg["cliente"].strip(),
103         "produto": reg["produto"].strip(),
104         "quantidade": quantidade,
105         "valor_unitario": valor_unitario,
106         "valor_total": round(quantidade * valor_unitario, 2),
107         "atualizado_em": atualizado_em.strftime("%Y-%m-%d %H:%M:%S"),
108         "cancelada": reg["status"].strip().upper() == "CANCELADA",
109     })
110
111     resumo["rejeitadas"] = len(rejeitados)
112
113     # R2: por id_venda, vale o atualizado_em mais recente

```

```

114     validas.sort(key=lambda v: v["atualizado_em"])
115     ultima = {v["id_venda"]: v for v in validas}
116     resumo["duplicatas"] = len(validas) - len(ultima)
117
118     vendas, cancelar = [], []
119     for v in ultima.values():
120         if v.pop("cancelada"):
121             cancelar.append((v["id_venda"], v["atualizado_em"]))
122         else:
123             vendas.append(v)
124     resumo["canceladas"] = len(cancelar)
125     resumo["carregadas"] = len(vendas)
126     return vendas, cancelar, rejeitados, resumo
127
128
129 def carregar(vendas, cancelar, rejeitados, arquivo, caminho_banco):
130     conn = sqlite3.connect(caminho_banco)
131     try:
132         conn.executescript(DDL)
133         with conn: # uma transação: commit no fim ou rollback se algo falhar
134             conn.execute(
135                 "DELETE FROM rejeitados WHERE arquivo = ?", (arquivo,)
136             )
137             conn.executemany(
138                 "INSERT INTO rejeitados VALUES (?, ?, ?)", rejeitados
139             )
140             conn.executemany(UPSERT, vendas)
141             # R1: canceladas saem, inclusive as de dias anteriores
142             conn.executemany(APAGAR_CANCELADA, cancelar)
143     finally:
144         conn.close()
145
146
147 def executar(caminho_csv, caminho_banco):
148     arquivo = Path(caminho_csv).name
149     df = extrair(caminho_csv)
150     vendas, cancelar, rejeitados, resumo = transformar(df, arquivo)
151     carregar(vendas, cancelar, rejeitados, arquivo, caminho_banco)
152     return resumo

```

Acertou as dez. Uma promessa falhou.

DEPOIS DA CARGA DE 02/09

```
resumo: lidas 8 = rodapé 1 + rejeitadas 1 + duplicatas 1
        + canceladas 1 + carregadas 4
vendas      1001  1002  1003  1006
1002        3299.0 (versão das 15:47)
rejeitados  linha 7: campo vazio: quantidade
```

02/09 DE NOVO, DEPOIS 03/09

```
02/09 de novo: 4 vendas, total 10075.02 (igual)
03/09: 1001  1002  1006  1007  1008 (1003 saiu)
```

TESTE A MAIS: 02/09 DE NOVO DEPOIS DE 03/09

```
vendas: 1001  1002  1003  1006  1007  1008 (1003 voltou)
```

O QUE ACONTECEU • 10 DE 10

- linhas 57-62 Lê acentos e separador e guarda tudo como texto, para conferir linha a linha.

- linha 71 Pula a linha de total.

- linhas 75-80 Campo vazio vai para a lista de rejeitados, com o motivo.

- linhas 33-45 Venda que já existe é atualizada, não duplicada.

- linha 142 Cancelada sai, até de dia anterior. Mas, ao apagar a venda, some a informação do cancelamento: é o erro do teste a mais.

Nota da aula: P2 27 → P3 31 (de 35).

As dez perguntas, programa por programa

PERGUNTA	P1	P2	P3
Abre o arquivo com acentos?	não	sim	sim
Ignora a linha de total?	não	não	sim
Centavos e totais certos?	não	sim	sim
02/09 é 2 de setembro?	não	sim	sim
Venda repetida: fica a mais recente?	não	sim	sim
Cancelada fica de fora?	não	sim	sim
Linha com defeito vai para uma lista?	não	não	sim
Rodar de novo não duplica?	não	não	sim
Dia novo mantém os anteriores?	não	sim	sim
Cancelada no dia seguinte sai?	não	não	sim
Acertos	0	6	10

NOTA DA AULA (7 CRITÉRIOS, MÁX. 35)



ONDE O PROMPT 3 FICOU PIOR

152 linhas: difícil para quem está começando.
E a resposta prometeu algo (A5) que o teste a mais derrubou.

Um bom prompt não é simplesmente um prompt longo. Ele precisa contar a situação real, como o dado chega e como a resposta vai ser usada, e dizer como o resultado vai ser conferido.

01

Mostre o dado como ele chega, com os defeitos.

02

Conte como a resposta vai ser usada.

03

Diga como você vai conferir.

04

Peça que a IA liste as dúvidas antes.

05

Confira com o dado real e teste as promessas.